

Predefined Functions

Predefined Functions

Plant Simulation provides the predefined functions listed in the table of contents to the left, which you can use in your controls.

Functions are not object-specific, they apply in general.

Operating System Functions

_Operating System Functions

SimTalk provides the functions listed in the table of contents to the left for accessing functions of the operating system.

availableMemory [SimTalk]

Returns the entire amount of main memory available in megabytes.

Type

Function

Syntax

```
availableMemory → real
```

Return Value

The return value has the data type *real*.

Example

```
print availableMemory
```

browseForFolder [SimTalk]

Opens a dialog that MS Windows provides, in which you can select the folder to be opened or create a new folder.

Type

Function

Syntax

```
browseForFolder(Message:string) → string
```

Parameter

The parameter *Message* of data type *string* is the message you want to show to the user.

Return Value

The return value has the data type *string*.

Example

```
browseForfolder("Select the folder you would like to open:")
```

copyFile [SimTalk]

Copies a file from one location to another location.

Remarks

If you deactivate the safety setting **File > Model Settings > General > Prohibit Access to the Computer**, the function *copyFile* can copy data from another folder and write it to the model folder or its sub-folders. The function cannot copy files from the model folder or its sub-folders.

Type

Function

Syntax

```
copyFile(Source:string, Destination:string) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Source* of data type *string* designates the source, i.e., the file you want to copy.
- The parameter *Destination* of data type *string* designates the destination, i.e., the folder into which you want to copy it.

Return Value

The return value has the data type *boolean*.

true if copying was successful.

false if it failed.

Example

```
copyFile("C:\file.txt", "C:\temp\file.txt")
```

See also

[Prohibit Access to the Computer \[model settings\]](#)

copyObjectsToClipboard [SimTalk]

Copies one or several objects to the internal *Plant Simulation* clipboard.

Remarks

Copying an object to the clipboard deletes the previous contents of the clipboard.

Type

Function

Syntax

```
copyObjectsToClipboard(Objects:object/object[])
```

Parameter

The parameter *Objects* of data type *object* can be a single object or an *array* of objects.

Note

A *Connector* can only be copied to the clipboard successfully if its predecessor object and its successor object are copied as well.

Example

```
var objs: object[]  
objs := [Station, Station1, Connector]  
copyObjectsToClipboard(objs)  
.Models.Frame2.pasteClipboard
```

See also

[pasteClipboard \[SimTalk\] - Frame](#)

copyTextToClipboard [SimTalk]

Copies the specified text to the clipboard.

Remarks

Copying text to the clipboard deletes the previous contents of the clipboard.

Type

Function

Syntax

```
copyTextToClipboard(TextToBeCopied:string)
```

Parameter

The parameter *TextToBeCopied* of data type *string* designates the text you want to copy.

Example

```
copyTextToClipboard("My long text, text, text.")
```

See also

[getTextFromClipboard \[SimTalk\]](#)

getApplicationProcessID [SimTalk]

Returns the current process identifier (PID) of the current *Plant Simulation* session.

Type

Function

Syntax

```
getApplicationProcessID → integer
```

Return Value

The return value has the data type *integer*.

Example

```
print getApplicationProcessID -- might return 8252 for the Plant Simulation  
process
```

See also

[system \[SimTalk\]](#)

[startExtProc \[SimTalk\]](#)

getCurrentDirectory [SimTalk]

Returns the current *Plant Simulation* working folder.

Type

Function

Syntax

```
getCurrentDirectory → string
```

Return Value

The return value has the data type *string*.

Example

```
print getCurrentDirectory
```

See also

[setCurrentDirectory \[SimTalk\]](#)

[Specifying Start Options, -cwd dir](#)

getEnv [SimTalk]

Returns the specified environment variable of the operating system.

Type

Function

Syntax

```
getEnv(EnvironmentVariable:string) → string
```

Parameter

The parameter *EnvironmentVariable* of data type *string* designates the environment variable.

Return Value

The return value has the data type *string*.

Is an empty string "" if the variable does not exist.

Example

```
print getEnv("PATH")
```

See also

[setEnv \[SimTalk\]](#)

getFilesOfFolder [SimTalk]

Returns a list of all files and folders which match the specified search pattern.

Type

Function

Syntax

```
getFilesOfFolder(SearchPattern:string) → list
```

Parameter

The parameter *SearchPattern* of data type *string* designates the directory and the search pattern.

The asterisk *** in the search pattern designates any string.

The question mark *?* in the search pattern designates a single character.

Return Value

The return value has the data type *list*.

Example

```
var allFiles, executables: list
allFiles := getFilesOfFolder("C:\Temp\*")           // returns a list of
all files and folders within the 'Temp' folder
executables := getFilesOfFolder("C:\Windows\*.exe") // returns a list of
all executables within the 'Windows' folder
```

getRegistry [SimTalk]

Returns a key from the folders HKEY_CLASSES_ROOT, HKEY_CURRENT_USER, and HKEY_LOCAL_MACHINE in the MS Windows Registry Editor.

Type

Function

Syntax

```
getRegistry(Key:string, Value:string)
```

Parameters

You can specify the following parameters:

- The parameter *Key* of data type *string* designates the key in a folder.
- The parameter *Value* of data type *string* designates the value of the key.

Return Values

The function returns values of these data types:

- *void* if the value does not exist
- *integer* for a DWORD value (REG_DWORD)
- *string* in all other cases (REG_SZ, REG_EXPAND_SZ, REG_MULTZI_SZ, REG_BINARY)

Example

```
print getRegistry("HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Tecnomatix Plant  
Simulation 2606", "")  
// returns the folder in which Plant Simulation is installed
```

getTextFromClipboard [SimTalk]

Copies text from the MS Windows clipboard and returns it as a *string*.

Type

Function

Syntax

```
getTextFromClipboard → string
```

Return Value

The return value has the data type *string*.

Example

```
print getTextFromClipboard // returns text from the Clipboard
```

See also

[copyTextToClipboard](#)

selectFileForOpen [SimTalk]

Opens the dialog **Open** in which the user can select the file he wants to open.

Remarks

If the user clicks **Cancel**, *Plant Simulation* returns an empty string "".

Type

Function

Syntax

```
selectFileForOpen([FileFilter:string]) → string
selectFileForOpen(["Text Files (*.txt)|
*.txt||"]) → string
selectFileForOpen(["Text Files (*.txt, *.doc)|
*.txt;*.doc||"]) → string
selectFileForOpen(["Model Files (*.spp, *.psobj)|*.spp;*.psobj|Text Files
(*.txt, *.doc)|*.txt;*.doc||"]) → string
selectFileForOpen([FileFilter:string,
PredefinedName:string]) →
string
```

Parameters

You can specify the following parameters:

- You can call *selectFileForOpen* with filters as parameters. These filters conform to the conventions set forth by Microsoft: `Comment1 | FileFilter1 | Comment2 | FileFilter2 ||`. Windows shows this **Comment**, which designates the type of file, in the text box **File type**. It only shows the types of files, whose extensions you entered as the **FileFilter**. If you want to show several file types, you have to separate the individual types with a pipe | sign. Terminate the string with two pipes ||.

To show in the text box File type	File Type
all text files, which have the extension .txt	<code>selectFileForOpen("Text Files (*.txt) *.txt ")</code>
all text files, which have the extension .txt and .doc	<code>selectFileForOpen("Text Files (*.txt, *.doc) *.txt;*.doc ")</code>
all files, which have the extension .spp, .psobj, .txt, and .doc	<code>selectFileForOpen("Model Files (*.spp, *.psobj) *.spp;*.psobj Text Files (*.txt, *.doc) *.txt;*.doc ")</code>

- The optional parameter *PredefinedName* of data type *string* designates the default path, i. e., the folder in which *Plant Simulation* opens the file selection box.

Return Value

The return value has the data type *string*.

Examples

```
var str: string
str := selectFileForOpen
if str /= ""
    statTable.readFile(str)
end
```

```
selectFileForOpen("", "C:\\Temp\\")
```

```
selectFileForOpen("Text Files (*.txt)|*.txt||", "C:\\Temp\\")
```

See also

[Open Model File](#)

selectFileForSave [SimTalk]

Opens the dialog **Save As** and lets you type in the file location.

Remarks

The user has to enter a name for the file to be saved into the text box **File Name** or select an existing name from the list, which the dialog shows.

Type

Function

Syntax

```
selectFileForSave([FileFilter:string, PredefinedName:string])    → string
selectFileForSave("Text Files (*.txt)|*.txt|")                  → string
selectFileForSave("Text Files (*.txt, *.doc)|*.txt;*.doc|")      → string
selectFileForSave("Model Files (*.spp, *.psobj)|*.spp;*.psobj|Text Files (*.txt, *.doc)|*.txt;*.doc|") → string
```

Parameters

You can specify the following parameters:

- The optional parameter *FileFilter* of data type *string* designates the file filter. These filters conform to the conventions set forth by Microsoft: **Comment1 | FileFilter1 | Comment2 | FileFilter2 |**. Windows shows this **Comment**, which designates the type of file, in the text box **File type**. It only shows the types of files, whose extensions you entered as the **FileFilter**. When you want to show several file types, you have to separate the individual types with a pipe **|** sign. Terminate the string with two pipes **| |**.

To show in the text box File type	File Type
all text files, which have the extension .txt	selectFileForSave("Text Files (*.txt) *.txt ")
all text files, which have the extension .txt and .doc	selectFileForSave("Text Files (*.txt, *.doc) *.txt;*.doc ")
all files, which have the extension .spp, .psobj, .txt, and .doc	selectFileForSave("Model Files (*.spp, *.psobj) *.spp;*.psobj Text Files (*.txt, *.doc) *.txt;*.doc ")

- The optional parameter *PredefinedName* of data type *string* designates the predefined file name. The dialog shows this name as the file name in the text box **File Name**. The user can overwrite it though.

Return Value

The return value has the data type *string*.

Is an empty string "" if the user clicks **Cancel**.

Examples

```
var str: string
str := selectFileForSave
if str /= ""
    saveModel(str)
end
```

```
selectFileForSave("Model Files (*.spp, *.psobj)|*.spp;*.psobj|Text Files  
(*.txt, *.doc)|*.txt;*.doc||", "MyModel.spp")
```

See also

[Save Model File As](#)

setCodePage [SimTalk]

Sets the code page for exchanging data in ANSI format via certain interfaces, for example the Oracle interface.

Type

Function

Syntax

```
setCodePage([CodePageName:integer])
```

Parameter

The parameter *CodePageName* of data type *integer* designates the number of the code page.

Return Value

The return value has the data type *integer*.

Is the previous value of the code page before it was changed.

Example

```
setCodePage(932)    // Japanese
setCodePage(936)    // Chinese
setCodePage(1250)   // Hungarian
setCodePage(1252)   // English, German
setCodePage(0)      // sets the code page of the operating system
print setCodePage   // returns the current code page
```

See also

[Specifying Start Options](#), /Codepage

setCurrentDirectory [SimTalk]

Sets the current working folder for *Plant Simulation*.

Remarks

If you deactivate the safety setting **File > Model Settings > General > Prohibit Access to the Computer** the function *setCurrentDirectory* can copy data from another folder and write it to the model folder or its sub-folders. The function cannot copy files from the model folder or its sub-folders.

Type

Function

Syntax

```
setCurrentDirectory(WorkingFolder:string) → boolean
```

Parameter

The parameter *WorkingFolder* of data type *string* designates the working folder.

Return Value

The return value has the data type *boolean*.

true if the working folder was set successfully.

false if the working folder was not set.

Example

```
print setCurrentDirectory("C:\users\hank")
```

See also

[Prohibit Access to the Computer \[model settings\]](#)

[getCurrentDirectory \[SimTalk\]](#)

[Specifying Start Options, -cwd dir](#)

setEnv [SimTalk]

Sets the specified environment variable.

Remarks

If you deactivate the safety setting **File > Model Settings > General > Prohibit Access to the Computer**, *Plant Simulation* does not run the function *setEnv* and shows an error message.

Type

Function

Syntax

```
setEnv(EnvironmentVariable:string, Value:string) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *EnvironmentVariable* of data type *string* designates the name of the variable.
- The parameter *Value* of data type *string* designates its value.

Return Value

The return value has the data type *boolean*.

Examples

```
setEnv("ralf","0")  
startExtProc("PlantSimulation2606.exe")
```

In the external process you just started you can then query the value you set:

```
print getEnv("ralf") // returns the value of the variable
```

See also

[Prohibit Access to the Computer \[model settings\]](#)

[getEnv \[SimTalk\]](#)

SHGetKnownFolderPath [SimTalk]

Returns the path of a standard folder in the file system of your computer.

Type

Function

Syntax

```
SHGetKnownFolderPath(CLSID:string) -> string
```

Parameter

The parameter *CLSID* of data type *string* designates a standard folder in the file system of your computer.

For more information, consult **KNOWNFOLDERID**.

Return Value

The return value has the data type *string*.

Example

```
print SHGetKnownFolderPath("{B4BFCC3A-DB2C-424C-B029-7FE99A87C641}")  
// returns the path to the Desktop, for example  
C:\Users\MyLoginName\Desktop
```

See also

<https://docs.microsoft.com/en-us/windows/win32/shell/knownfolderid>

sleep [SimTalk]

Suspends a *Method* for the specified time. The time is not the simulation time of the *EventController*, but the real-time of your computer's clock.

Type

Function

Syntax

```
sleep(Time:real[, SuspendProcess:boolean:=true])
```

Parameters

You can specify the following parameters:

- The parameter *Time* of data type *real* designates the amount of time in seconds in real-time for which the *Method* will be suspended.
- The optional parameter *SuspendProcess* of data type *boolean* determines if the function *sleep* suspends the *Plant Simulation* process for the time in seconds designated by the parameter of data type *real* (*true*). This is especially helpful when you are starting other processes from *Plant Simulation* and you do not want *Plant Simulation* to consume CPU time.

If you enter *false*, *Plant Simulation* will not be blocked for the time span, which you entered, but only stops executing the *Method*. This way *sleep* works like a **wait-instruction**. The only difference is that the time during which *sleep* waits is real-time, while the time for *wait* is simulation time.

Default Value of the Parameter

The default value is *true*.

Example

```
sleep(3.5, false)
```

See also

[wait \[SimTalk\]](#)

startExtProc [SimTalk]

Starts the specified external process.

Remarks

If you deactivate the safety setting **File > Model Settings > General > Prohibit Access to the Computer**, *Plant Simulation* does not execute the function *startExtProc* and shows an error message.

Type

Function

Syntax

```
startExtProc(PathToProgram:string[, Visible:boolean,
WaitUntilProcessTerminated:boolean]) → integer
```

Parameters

You can specify the following parameters:

- The parameter *PathToProgram* of data type *string* designates the external process.
This external process usually is a program with a graphical user interface that does not have a *Console*. *Plant Simulation* itself will continue to be executed while the external process is open.
Enter a double-backslash `\\` as a path separator. The function either returns the process ID (PID) or it returns 0 if the function fails.
- The parameter *Visible* sets if the started program is to be visible (`true`) or not (`false`).
If you do not specify the parameter, the program window will be visible.
- The parameter *WaitUntilProcessTerminated* sets if the function is to wait until the started program was terminated (`true`) or not (`false`).
If you do not specify the parameter, the function does not wait, i. e., the instructions, which are to follow after *startExtProc*, will be executed immediately.

Note

If the command window distracts you from a system command, for example `system("dir file.txt")`, you can hide the command window with the function *startExtProc*, for example `startExtProc("cmd.exe /C dir file.txt", false, true)`

Return Value

The return value has the data type *integer*.

Example

```
startExtProc("C:\\Program Files (x86)\\Adobe\\Acrobat Reader DC\\Reader\\  
\\AcroRd32.exe")
```

See also

[Prohibit Access to the Computer \[model settings\]](#)

[system \[SimTalk\]](#)

[getApplicationProcessID \[SimTalk\]](#)

system [SimTalk]

Executes the specified system command during the simulation run. *Plant Simulation* will be blocked until the system command is completely executed.

Remarks

Note

You can only specify DOS commands. Always start programs with a graphical user interface using the function *startExtProc* or programs will block each other.

If you deactivate the safety setting **File > Model Settings > General > Prohibit Access to the Computer**, *Plant Simulation* does not execute the function *system* and shows an error message.

Type

Function

Syntax

```
system(Command:string) → integer
```

Parameter

The parameter *Command* of data type *string* designates the command.

Return Value

The return value has the data type *integer*.

Is identical with the exit state of the command.

Example

```
if system("del C:\temp\file.txt") = 0  
  print "file deleted"  
end
```

See also

[Prohibit Access to the Computer \[model settings\]](#)

[startExtProc \[SimTalk\]](#)